

Course Project in Object-Oriented Programming

Veterinary Practice Management System

(VPMS)

Subject:	Object-Oriented Programming
Language:	C++17, Qt 6 (Widgets)
Testing:	Qt Test — 48 tests, 0 failed
Persistence:	File I/O only (no database)

2026

Table of Contents

1. Introduction and Project Goals
2. System Architecture
3. Core Module — Persistence and Infrastructure
4. Animals & Owners Module
5. Visits & Services Module
6. Inventory & Hospitalization Module
7. Graphical User Interface (Qt Widgets)
8. OOP Concepts Demonstrated
9. Class Diagrams
10. Testing
11. Build and Run Instructions

1. Introduction and Project Goals

VPMS (Veterinary Practice Management System) is a desktop application that automates the day-to-day operations of a veterinary clinic. It manages clients, animals, medical visits, service catalog, inventory stock, and hospitalization records.

Project objectives:

- Demonstrate core OOP principles: inheritance, polymorphism, operator overloading, encapsulation;
- Implement full data persistence without any database — using C++ file streams only;
- Apply design patterns: Facade, Factory Method, Template Repository;
- Build a clean Qt Widgets GUI strictly separated from business logic;
- Cover all business logic with automated unit tests.

Key constraint: No database systems (SQLite, PostgreSQL, etc.) are used anywhere in the project. All persistence is achieved through `std::fstream` and per-entity `serialize` / `deserialize` methods.

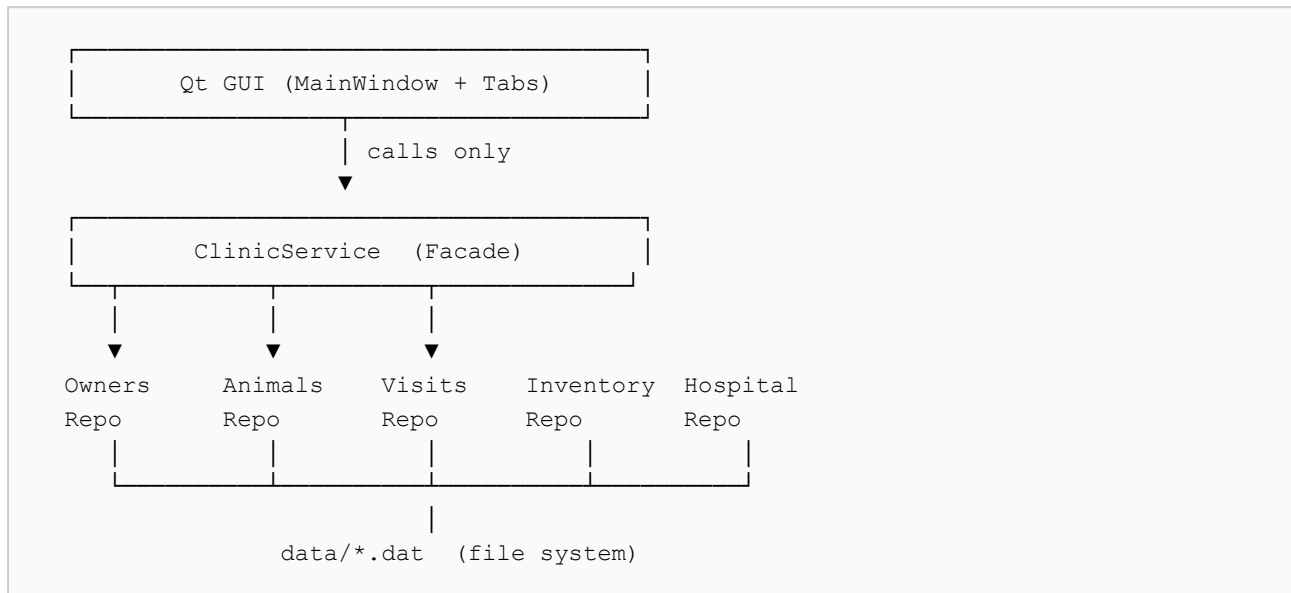
2. System Architecture

The system is divided into four domain modules plus a GUI layer:

Module	Responsibility	Key Classes
Core	Persistence, ID generation, exceptions	Repository<T>, IdGenerator, exception hierarchy
Animals & Owners	Animal hierarchy, client records	Animal, Dog, Cat, Bird, Reptile, Owner, AnimalFactory
Visits & Services	Visit records, service catalog	Visit, Service, Consultation, Vaccination, Surgery, Grooming, ServiceFactory
Inventory & Hospitalization	Stock management, inpatient records	InventoryItem, HospitalizationRecord

GUI	Qt Widgets interface	MainWindow, ClientsTab, VisitsTab, ServicesTab, InventoryTab, HospitalizationTab
------------	----------------------	--

Layer Interaction



ClinicService is the central facade. It owns all repositories and exposes a unified API. GUI code never touches repositories or files directly.

3. Core Module — Persistence and Infrastructure

3.1 ISerializable

Every persistent entity must implement this interface to be stored in a repository:

```
class ISerializable {
public:
    virtual int  getId() const = 0;
    virtual void serialize(ostream& out) const = 0;
    virtual void deserialize(istream& in) = 0;
    virtual ~ISerializable() = default;
};
```

3.2 Repository<T>

A generic template class that implements file-backed CRUD for any `ISerializable` type. It keeps all objects in memory and persists them to disk on every write operation.

```
template <typename T>
class Repository {
public:
    void add(const T& item);           // throws DuplicateIdException
    T&  getId(int id);                 // throws NotFoundException
    void remove(int id);
    void update(const T& item);
    bool exists(int id) const;
    int  count() const;
protected:
    void load(); // called on construction — reads file
    void save(); // called after every mutation — rewrites file
};
```

3.3 IdGenerator

Generates auto-incrementing IDs per entity type (`owner`, `animal`, `visit`, ...). Counters are persisted in `data/ids.dat` so IDs remain unique across application restarts.

3.4 Exception Hierarchy

Exception Class	When Thrown
NotFoundException	Requested ID does not exist in the repository
DuplicateIdException	Attempt to add an object with an already-used ID
ValidationException	Business rule violated (empty name, negative price, etc.)
InsufficientStockException	Stock-out request exceeds available quantity
FileIOException	Cannot open a data file for reading or writing

4. Animals & Owners Module

4.1 Animal Hierarchy

`Animal` is an abstract base class. Each concrete subclass overrides two pure virtual methods:

```
class Animal : public ISerializable {
public:
    virtual string getSpeciesInfo() const = 0;
    virtual double calculateTreatmentCost() const = 0;
    virtual void serialize(ostream&) const override;
    virtual void deserialize(istream&) override;
};
```

Class	Extra Field	calculateTreatmentCost()	getSpeciesInfo() example
Dog	_breed	80.0	"Dog - Breed: Labrador"
Cat	_furType	60.0	"Cat - Fur: Persian"
Bird	_species	40.0	"Bird - Species: Parrot"
Reptile	_reptileType	50.0	"Reptile - Type: Iguana"

4.2 AnimalFactory

Implements the Factory Method pattern. Creates the correct subclass from a string type tag:

```
Animal* AnimalFactory::create(const string& typeTag, int id,
                              const string& name, int age,
                              int ownerId, const string& extra) {
    if (typeTag == "DOG")    return new Dog(id, name, age, ownerId, extra);
    if (typeTag == "CAT")    return new Cat(id, name, age, ownerId, extra);
    if (typeTag == "BIRD")   return new Bird(id, name, age, ownerId, extra);
    if (typeTag == "REPTILE") return new Reptile(id, name, age, ownerId, extra);
    throw ValidationException("Unknown animal type: " + typeTag);
}
```

4.3 Owner

Stores client data: name and contact information. Implements `ISerializable` and overloads `operator==` (equality by ID).

5. Visits & Services Module

5.1 Service Hierarchy

Each service type inherits `Service` and overrides its pricing logic:

```
class Service : public ISerializable {
public:
    virtual double getFinalPrice() const = 0;
    virtual string getDescription() const = 0;
    virtual void    applyDiscount(double pct);
};
```

Class	getFinalPrice() formula	Notable behaviour
Consultation	= basePrice	Extra field: duration in minutes
Vaccination	= basePrice	Extra field: vaccine type string
Surgery	basePrice × 1.2 (with anesthesia) basePrice × 1.0 (without)	Discount capped at 10%; overrides applyDiscount()
Grooming	Premium: × 1.15 Basic: × 0.90 Standard: × 1.0	Service level affects the final price

5.2 Visit

Records the link between an animal, its owner, and a date, along with a list of service IDs and the computed total cost. Two comparison operators are overloaded:

```
bool Visit::operator<(const Visit& other) const { return _date < other._date; }
bool Visit::operator==(const Visit& other) const { return _id == other._id; }
```

6. Inventory & Hospitalization Module

6.1 InventoryItem

Represents a stock item: name, quantity, unit price, and category. Stock-in and stock-out operations are expressed via overloaded operators:

```
InventoryItem& InventoryItem::operator+=(int qty) { _quantity += qty; return *this; }  
InventoryItem& InventoryItem::operator-=(int qty) { _quantity -= qty; return *this; }
```

Items with quantity ≤ 10 are highlighted in red in the GUI as a low-stock warning.

6.2 HospitalizationRecord

Tracks an animal's hospital stay: ward, admission date, discharge date (empty = still active), daily rate, accumulated bill, and any services rendered during the stay.

```
bool HospitalizationRecord::isActive() const {  
    return _dischargeDate.empty();  
}
```

On discharge, the system computes `days × dailyRate + servicesCost` and stores it as the total bill.

7. Graphical User Interface (Qt Widgets)

The interface is built around a five-tab `QTabWidget`. GUI code calls **only** `ClinicService` — no widget has any knowledge of repositories, files, or raw pointers.

Tab	Functionality
Clients	Three-panel QSplitter: owner list → animal table → visit table. Full CRUD for clients and animals.
All Visits	All clinic visits with client filter. New-visit dialog with per-owner animal dropdown and checkbox service selection.
Services	Service catalog with type filter. Add-service dialog shows dynamic extra fields depending on the selected type.
Inventory	Stock table. Low-stock rows highlighted red. Stock-in and stock-out via QInputDialog.
Hospitalization	Separate active/discharged sections. Current estimated bill computed from today's date in real time.

8. OOP Concepts Demonstrated

8.1 Inheritance

Two independent inheritance hierarchies are used:

- `Animal` → `Dog`, `Cat`, `Bird`, `Reptile`
- `Service` → `Consultation`, `Vaccination`, `Surgery`, `Grooming`

Both hierarchies implement `ISerializable`, allowing heterogeneous collections of objects to be stored and loaded through a single repository mechanism.

8.2 Polymorphism

The clearest example is price calculation in `ClinicService::createVisit()`. The method does not know the concrete type of each service — it calls the virtual method through a base-class

pointer:

```
double total = 0.0;
for (int sid : serviceIds) {
    Service* s = _services->getById(sid);
    total += s->getFinalPrice();    // polymorphic dispatch
}
```

Similarly, when saving animals, the repository calls `a->serialize(file)` on every `Animal*`, and each subclass writes its own extra field (breed, fur type, etc.) to the file.

8.3 Operator Overloading

Class	Operator	Meaning
InventoryItem	<code>+=</code> , <code>-=</code>	Stock-in and stock-out operations
Visit	<code><</code>	Sort visits by date (used with <code>std::sort</code>)
Visit	<code>==</code>	Identity comparison by ID
Owner, InventoryItem, HospitalizationRecord	<code>==</code>	Identity comparison by ID
Visit, InventoryItem, HospitalizationRecord	<code><<</code>	Stream output (diagnostic / logging)

8.4 Exception Handling

All domain errors are thrown as subclasses of `VPMSException`. The GUI catches them at the boundary and displays a user-friendly message:

```
// ClinicService - throws:
if (age <= 0)
    throw ValidationException("Animal age must be positive.");
if (!_owners.exists(ownerId))
    throw NotFoundException(ownerId);

// GUI - catches at the boundary:
try {
    _svc.createVisit(animalId, ownerId, date, serviceIds);
    loadVisits();
} catch (const exception& e) {
```

```

    QMessageBox::warning(this, "Error", e.what());
}

```

8.5 File I/O and Serialization

Every entity implements `serialize` / `deserialize`. Records are delimited by a `---` sentinel line.

Example — Dog record on disk:

```

DOG          ← type tag (consumed by AnimalRepository before calling
deserialize)
1            ← ID
Rex          ← name
5            ← age
2            ← owner ID
German Shepherd ← breed (Dog-specific field)
---          ← sentinel

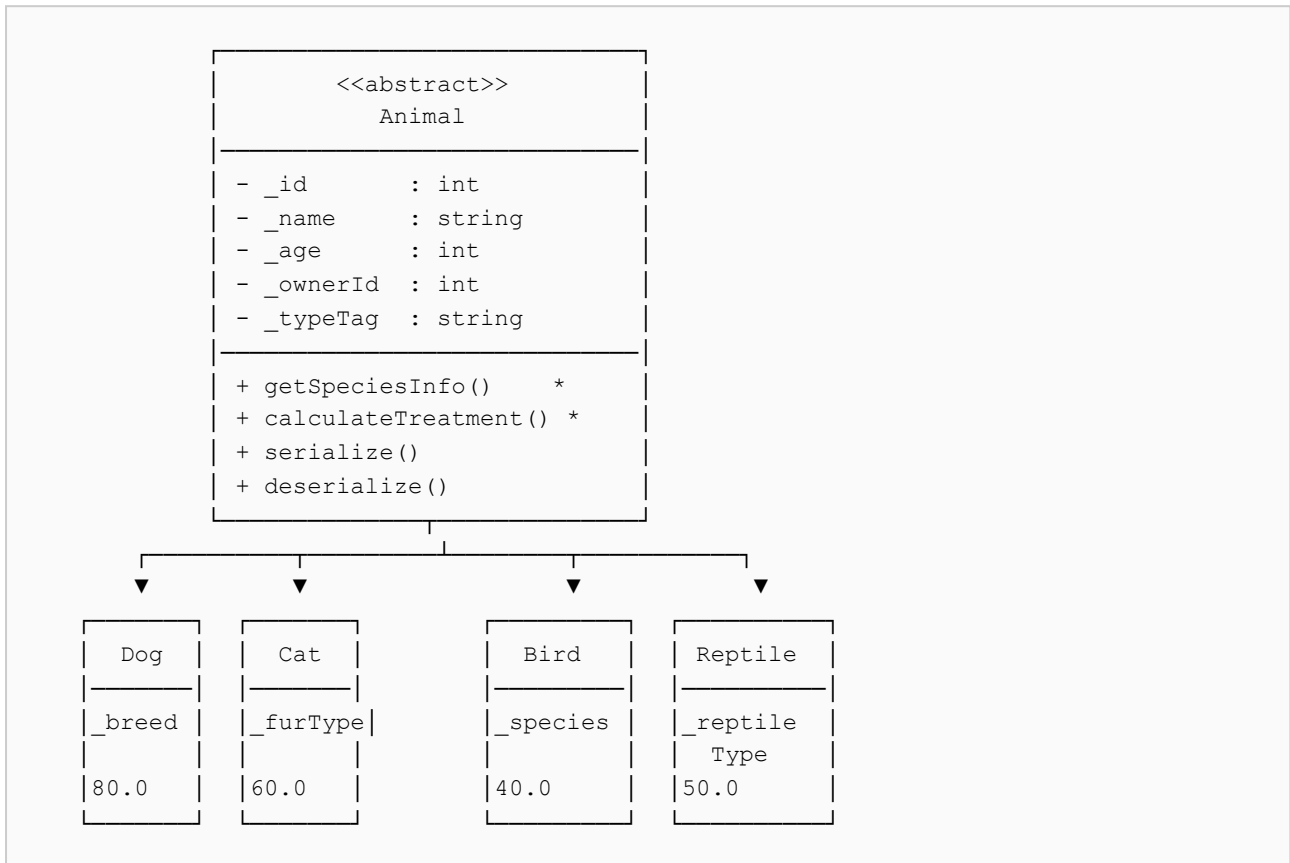
```

8.6 Design Patterns

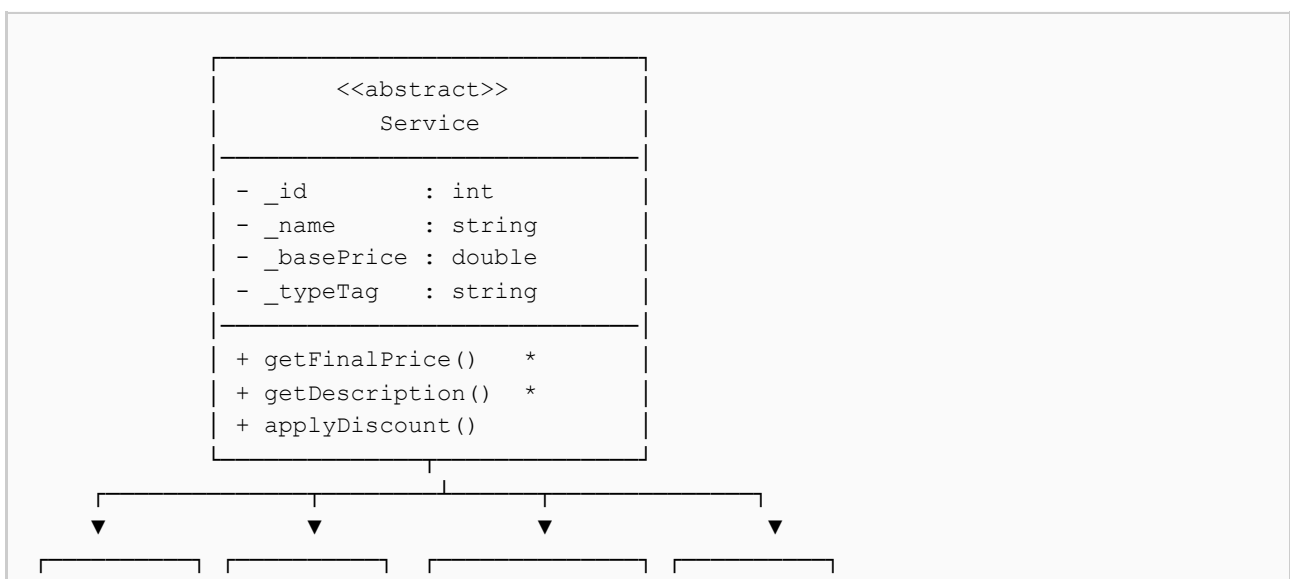
Pattern	Implementation
Facade	<code>ClinicService</code> hides all repositories behind a single high-level API
Factory Method	<code>AnimalFactory::create()</code> and <code>ServiceFactory::create()</code> instantiate the right subclass from a string tag
Template Method	<code>Repository<T>::load()</code> / <code>save()</code> define the algorithm; <code>AnimalRepository</code> and <code>ServiceRepository</code> override the polymorphic steps

9. Class Diagrams

9.1 Animal Hierarchy

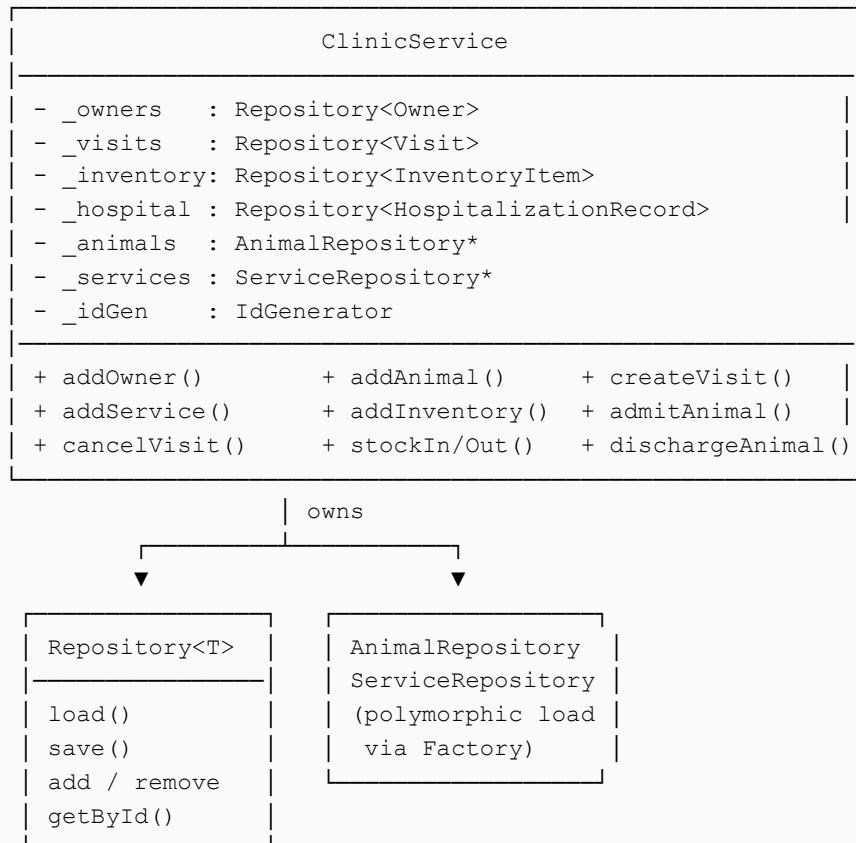


9.2 Service Hierarchy



Consultat.	Vaccinat.	Surgery	Grooming
<u>_duration</u>	<u>_vaccine</u>	<u>_surgeryType</u>	<u>_grooming</u>
Minutes	Type	_reqAnesthesia	Level
= base	= base	×1.2 or ×1.0	×1.15/0.9

9.3 Repository and ClinicService



10. Testing

Tests are written with the **Qt Test** framework in a standalone project (`tests/tests.pro`). All tests are fully isolated — file-based tests use `/tmp/` and clean up after themselves via the `cleanup()` slot.

Test Class	What Is Tested	Count
TestAnimals	Animal hierarchy, Owner, AnimalFactory, serialization round-trips, polymorphic dispatch	13
TestVisits	Service pricing (all 4 types), Surgery discount cap, Visit comparison operators, ServiceFactory	15
TestInventory	InventoryItem operators, HospitalizationRecord active/discharge logic, serialization	10
TestCore	Repository CRUD, duplicate/not-found exceptions, file persistence, IdGenerator continuity	10
Total		48

Test Output

```
***** Start testing of TestAnimals *****
PASS   : TestAnimals::dogGetters()
PASS   : TestAnimals::dogSpeciesInfo()
PASS   : TestAnimals::dogTreatmentCost()
PASS   : TestAnimals::catTreatmentCost()
PASS   : TestAnimals::birdTreatmentCost()
PASS   : TestAnimals::reptileTreatmentCost()
PASS   : TestAnimals::polymorphicDispatch()
PASS   : TestAnimals::dogSerializeDeserialize()
PASS   : TestAnimals::catSerializeDeserialize()
PASS   : TestAnimals::ownerGettersAndSetters()
PASS   : TestAnimals::ownerEquality()
PASS   : TestAnimals::animalFactoryType()
PASS   : TestAnimals::animalFactoryUnknownThrows()
Totals: 15 passed, 0 failed, 0 skipped
```

***** Start testing of TestVisits *****

PASS : TestVisits::consultationPrice()
PASS : TestVisits::consultationDescription()
PASS : TestVisits::vaccinationPrice()
PASS : TestVisits::surgeryWithAnesthesia()
PASS : TestVisits::surgeryWithoutAnesthesia()
PASS : TestVisits::surgeryDiscountCap()
PASS : TestVisits::groomingPremium()
PASS : TestVisits::groomingBasic()
PASS : TestVisits::groomingStandard()
PASS : TestVisits::visitGetters()
PASS : TestVisits::visitOrderByDate()
PASS : TestVisits::visitEquality()
PASS : TestVisits::consultationSerializeDeserialize()
PASS : TestVisits::serviceFactoryTypes()
PASS : TestVisits::serviceFactoryUnknownThrows()
Totals: 17 passed, 0 failed, 0 skipped

***** Start testing of TestInventory *****

PASS : TestInventory::inventoryGetters()
PASS : TestInventory::stockIn()
PASS : TestInventory::stockOut()
PASS : TestInventory::inventoryEquality()
PASS : TestInventory::inventoryOrder()
PASS : TestInventory::inventorySerializeDeserialize()
PASS : TestInventory::hospitalizationGetters()
PASS : TestInventory::hospitalizationIsActive()
PASS : TestInventory::hospitalizationDischarge()
PASS : TestInventory::hospitalizationSerializeDeserialize()
Totals: 12 passed, 0 failed, 0 skipped

***** Start testing of TestCore *****

PASS : TestCore::repositoryAdd()
PASS : TestCore::repositoryDuplicate()
PASS : TestCore::repositoryNotFound()
PASS : TestCore::repositoryRemove()
PASS : TestCore::repositoryExists()
PASS : TestCore::repositoryCount()
PASS : TestCore::repositoryPersistence()
PASS : TestCore::exceptionMessages()
PASS : TestCore::idGeneratorNext()
PASS : TestCore::idGeneratorPersistence()
Totals: 12 passed, 0 failed, 0 skipped

```
TOTAL: 48 passed, 0 failed, 0 skipped
```

11. Build and Run Instructions

Requirements

- Qt 6.x (modules: core, gui, widgets, testlib)
- C++17-compatible compiler (Clang or GCC)
- qmake

Build the main application

```
cd vpms/  
qmake vpms.pro  
make -j$(nproc)  
open vpms.app           # macOS
```

Build and run tests

```
cd tests/  
qmake tests.pro  
make -j$(nproc)  
./vpms_tests
```

Directory Structure

```
oop project/  
├─ vpms/  
│   ├── core/           # Repository<T>, IdGenerator, exceptions  
│   ├── animals/        # Animal hierarchy, Owner, AnimalFactory  
│   ├── visits/         # Service hierarchy, Visit, ServiceFactory  
│   ├── inventory/      # InventoryItem, HospitalizationRecord  
│   ├── gui/            # Qt Widgets (MainWindow + 5 tabs)  
│   ├── data/           # Persistent data files (*.dat)  
│   └─ vpms.pro  
└─ tests/
```

```
|— tests.cpp      # 48 Qt Test cases
|— tests.pro
```